



KG COLLEGE OF ARTS AND SCIENCE

Autonomous Institution | Affiliated to Bharathiar University

Accredited with A++ Grade by NAAC

ISO 9001:2015 Certified Institution

KGiSL Campus, Saravanampatti, Coimbatore – 641 035

LAB MANUAL

CORE LAB - VI: LINUX AND SHELL PROGRAMMING

B.Sc. CS / BCA / B.Sc. CT / B.Sc. IT / B.Sc. AI & DS

2024 BATCH

LIST OF EXPERIMENTS

COURSE CODE : 24BCT54P

COURSE TITLE : CORE LAB - VI: LINUX AND SHELL PROGRAMMING

S. No.	List of Programs
1	Write a shell script to stimulate the file commands : rm, cp, cat, mv, cmp, wc, split, diff.
2	Write a shell script to show the following system configuration: • Currently logged user and his log name • Current shell, home directory, Operating System type, current Path setting, current working directory • Show currently logged number of users, show all available shells • Show CPU information like processor type, speed • Show memory information
3	Write a shell script to implement the following: pipes, Redirection and tee commands.
4	Write a shell script for displaying current date, user name, file listing and directories by getting user choice.
5	Write a shell script to implement the filter commands.
6	Write a shell script to remove the files which has file size as zero bytes.
7	Write a shell script to find the sum of the individual digits of a given number.
8	Write a shell script to find the greatest among the given set of numbers using command line arguments.
9	Write a shell script for palindrome checking.
10	Write a shell script to print the multiplication table of the given argument using for loop.
11	Write a shell script to display and analyze group information. - Display all groups available in the system. - Display the group ID of a specified group. - List users belonging to a particular group. - Count the total number of groups available. - Display groups assigned to the current user.
12	Write a shell script to monitor process activities. - Display all running processes. - Display processes belonging to the current user. - Show top five CPU consuming processes. - Display process ID and parent process ID. - Display the total number of running processes.

13	Write a shell script to display network configuration information. a. Display system hostname. b. Display IP address of all network interfaces. c. Display routing table information. d. Display DNS server configuration. e. Test network connectivity with a remote host.
14	Write a shell script to monitor system logs and detect suspicious activity. a. Display recent system log entries. b. Display login history of users. c. Display failed login attempts. d. Search for specific keywords in log files. e. Display the last 10 security related log messages.
15	Write a shell script to analyze disk usage. a. Display disk usage of all directories in the home folder. b. Display the top 5 largest directories in the system. c. Display the number of files in each directory. d. Display the file system type of each partition. e. Display free disk space available.
	Total Hours 75
Text Books	
1.	NIIT. (2006), Operating System Linux, Eastern Economy Edition, PHI Learning.
2.	Venkateswarlu, N.B. (2016), Introduction to Linux: Installation and Programming, 1 st Edition, BS Publications.
3.	Christopher Negus C. (2025), Linux Bible, 10th Edition, Wiley publishers.
Reference Books	
1.	Petersen, R. (2008), Linux: The Complete Reference, 6 th Edition, Tata McGraw-Hill Publishing Company Limited, New Delhi.
2.	Soyinka, W. (2015), Linux Administration: A Beginner's Guide, McGraw-Hill Education.
3.	Jones, M. T. (2009), Linux System Programming and Administration, Jones & Bartlett Learning.
Web Resources(Swayam/NPTEL/Others)	
1.	https://www.classcentral.com/course/swayam-linux-applications-in-engineering-education-503682?
2.	https://www.redhat.com/en/services/training/getting-started-with-linux-fundamentals?
3.	https://developers.redhat.com/

Sample Program

PROGRAM 1:

Write a shell script to stimulate the file commands: rm, cp, cat, mv, cmp, wc, split, diff.

AIM

To demonstrate basic Linux file manipulation commands: rm, cp, cat, mv, cmp, wc, split, and diff.

PROCEDURE

STEP 1: Start the process.

STEP 2: In the Linux command prompt, perform operations using file commands.

STEP 3: Use vi editor to create files: vi filename. Use mkdir to create directories.

STEP 4: Use cat command to create (cat >filename), append (cat >>filename), and display (cat filename) files.

STEP 5: Use wc command to count lines, characters and bytes: wc filename.

STEP 6: Use cp to copy a file: cp oldfile newfile.

STEP 7: Use cmp to compare two files, rm to remove a file, mv to move/rename a file.

STEP 8: Use split to split a file into parts. Use diff to show differences between two files.

STEP 9: Type all commands in a shell script file (vi filename.sh) and execute: sh filename.sh.

STEP 10: Stop the process.

SHELL SCRIPT

```
#!/bin/bash
ls                # To list all files in the directory
cp file1 file2    # To copy a file
cat file2         # To view contents of file2
mv file3 file4    # To move/rename a file
cat file4         # To display contents of file4
wc file1          # To see word/line/char count
wc -l file1       # To count only lines
wc -w file1       # To count only words
wc -c file1       # To count only characters
split -3 file1    # To split file into parts of 3 lines
head -3 file1     # To display first 3 lines
tail -3 file1     # To display last 3 lines
cmp file1 file2   # To compare two files byte by byte
diff file1 file2  # To show line-by-line differences
rm file6          # To remove file6
rm -r *           # To remove all files in the directory
```

OUTPUT

```
[ita1@Kgcaslinux ~]$ ls
file1 file2 sun
[ita1@Kgcaslinux ~]$ cp file1 file3
[ita1@Kgcaslinux ~]$ cat file3
a b c d e f g
[ita1@Kgcaslinux ~]$ mv file2 file4
[ita1@Kgcaslinux ~]$ wc file4
 7  7 28 file4
[ita1@Kgcaslinux ~]$ split -3 file4
[ita1@Kgcaslinux ~]$ ls
file1 file3 file4 sun xaa xab xac
[ita1@Kgcaslinux ~]$ cmp file1 file4
file1 file4 differ: byte 1, line 1
[ita1@Kgcaslinux ~]$ diff file1 file4
1,7c1,7
< a < b < c ...
---
> sun > mon > tue ...
[ita1@Kgcaslinux ~]$ rm xac
[ita1@Kgcaslinux ~]$ rm -r *
```

PROGRAM 2:

Write a shell script to show the following system configuration:

- Currently logged user and his log name
- Current shell, home directory, Operating System type, current Path setting, current working directory
- Show currently logged number of users, show all available shells
- Show CPU information like processor type, speed
- Show memory information

AIM

To write a shell script that displays user and system information including logged-in user, shell, OS type, path, working directory, CPU and memory details.

PROCEDURE

STEP 1: Start the process.

STEP 2: Use logname command to check login name of user; use \$SHELL to check current shell.

STEP 3: Use \$OSTYPE environment variable to display the OS type.

STEP 4: Use \$PATH to check the path settings for directories.

STEP 5: Use pwd command to view the present working directory.

STEP 6: Use lscpu command to check CPU information.

STEP 7: Stop the process.

SHELL SCRIPT

```
#!/bin/bash
echo -e "Username      : $USER"
echo -e "Login Name       : $LOGNAME"
echo -e "Current Shell    : $SHELL"
echo -e "Home Directory   : $HOME"
echo -e "OS Type          : $OSTYPE"
echo -e "Current Path     : $PATH"
echo -e "Working Dir      : `pwd`"
echo -e "Logged-in Users  : `who | wc -l`"
echo -e "Available Shells:"
cat /etc/shells
echo -e "CPU Info         : "
lscpu | grep -E 'Model name|CPU MHz'
echo -e "Memory Info      : "
free -m
```

OUTPUT

```
[ital@Kgcaslinux ~]$ sh sysinfo.sh
Username      : tsa
Login Name    : tsa
Current Shell : /bin/bash
Home Directory : /home/tsa
OS Type       : linux-gnu
Current Path  : /usr/local/bin:/usr/bin:/bin
Working Dir   : /home/tsa
Logged-in Users : 2
Available Shells:
/bin/sh /bin/bash /usr/bin/zsh
CPU Info      :
Model name: Intel(R) Core(TM) i5-8250U CPU @ 1.60GHz
Memory Info   :
              total  used  free
Mem:          7892  3400  4492
```

PROGRAM 3:

Write a shell script to implement the following: pipes, Redirection and tee commands.

AIM

To write a shell script that demonstrates the use of pipe (|), I/O redirection (>, >>, <), more command, and tee command.

PROCEDURE

STEP 1: Start the process.

STEP 2: Use the pipe command (|) to link one command's output to another command's input.

STEP 3: Use output redirection (>) to write output to a file, and append redirection (>>) to add to a file.

STEP 4: Use the more command to display output page by page on the screen.

STEP 5: Use the tee command to display output on screen and save it to a file simultaneously.

STEP 6: Save and end the process.

SHELL SCRIPT

```
#!/bin/bash

# Pipe (|): pass output of ls to wc
echo "--- Pipe: Count files using ls | wc ---"
ls -la | wc

# Output Redirection (>): write to file
echo "--- Redirection (>): Write to file ---"
echo "Welcome to Linux" > sample1.txt
cat sample1.txt

# Append Redirection (>>): add to existing file
echo "--- Append (>>): Add to file ---"
echo "Always best learning institute" >> sample1.txt
cat sample1.txt

# More: paginated output
echo "--- More: Display files page by page ---"
ls -la | more

# Tee: output to screen AND save to file
echo "--- Tee: Screen + Save to file ---"
ls -la | tee tee_output.txt
echo ""
echo "Content saved in tee_output.txt:"
cat tee_output.txt
```

OUTPUT

```
[tsa03@localhost ~]$ sh pipe_redirect.sh
--- Pipe: Count files using ls | wc ---
64  569 3333

--- Redirection (>): Write to file ---
Welcome to Linux

--- Append (>>): Add to file ---
Welcome to Linux
Always best learning institute

--- More: Display files page by page ---
total 504
drwx----- 4 tsa03 tsa03 4096 Feb 13 08:40 .
-rw-rw-r-- 1 tsa03 tsa03  24 Jan 23 aa

--- Tee: Screen + Save to file ---
-rw-r--r-- 1 tsa03 tsa03  24 Jun 3 sample1.txt
Content saved in tee_output.txt:
-rw-r--r-- 1 tsa03 tsa03  24 Jun 3 sample1.txt
```

PROGRAM 4:

Write a shell script for displaying current date, user name, file listing and directories by getting user choice.

AIM

To write a menu-driven shell script that displays current date, username, file listing, and directories based on the user's choice using a case statement.

PROCEDURE

STEP 1: Start the process.

STEP 2: Use the case statement to perform different actions from a single prompt.

STEP 3: Declare case variables and case command for the program.

STEP 4: If choice is 1, display current user (whoami). If choice is 2, display logged-in users (who).

STEP 5: If choice is 3, display date. If choice is 4, display calendar. If choice is 5, display current directory. If choice is 6, display file listing.

STEP 6: If none of the choices is matched, the default case gets executed.

STEP 7: Stop the process.

SHELL SCRIPT

```
#!/bin/bash

echo "1- Who am I?"
echo "2- Who is logged on?"
echo "3- Date"
echo "4- Calendar"
echo "5- Current Directory"
echo "6- File Listing"
echo "Enter your choice:"
read n

case $n in
    1) whoami ;;
    2) who ;;
    3) date ;;
    4) cal ;;
    5) pwd ;;
    6) ls ;;
    *) echo "Invalid choice" ;;
esac
```

OUTPUT

```
[ita1@Kgcslinux ~]$ sh menu.sh
1- Who am I?
2- Who is logged on?
3- Date
4- Calendar
5- Current Directory
6- File Listing
Enter your choice:
2
tsa pts/0 Jun 3 09:00 (192.168.1.10)

Enter your choice:
3
Mon Jun 3 09:15:30 IST 2026
```

PROGRAM 5:

Write a shell script to implement the filter commands.

AIM

To write a shell script that demonstrates Linux filter commands: grep, wc, tr, and cut using the /etc/passwd file.

PROCEDURE

STEP 1: Start the process.

STEP 2: Create a file using vi editor.

STEP 3: Copy /etc/passwd to a local file: cp /etc/passwd passwd.

STEP 4: Use grep -n 'root' passwd to display lines containing the word root with line numbers.

STEP 5: Use grep -c 'root' passwd to count the number of lines containing root.

STEP 6: Use wc passwd to display number of lines, words, and characters.

STEP 7: Use grep -v 'root' passwd to display all lines that do NOT match root.

STEP 8: Use tr ':' '*' <passwd to replace ':' with '*' in the file.

STEP 9: Use cut -d':' -f1 passwd to display only the first column (usernames).

STEP 10: Display the output on screen.

STEP 11: Stop the process.

SHELL SCRIPT

```
#!/bin/bash

echo "Filter Commands Demo"
cp /etc/passwd passwd      # Copy /etc/passwd to local file

echo "Lines, words and characters in passwd:"
wc passwd

echo "Lines containing the word root:"
grep -n "root" passwd | more

echo "Count of lines containing root:"
grep -c "root" passwd

echo "Lines that do NOT contain root:"
grep -v "root" passwd | more

echo "Replace ':' with '*' in passwd:"
tr ':' '*' < passwd | more

echo "First column (usernames) of passwd:"
cut -d':' -f1 passwd
```

OUTPUT

```
[cs2ax@kgcaslinux ~]$ sh filter.sh
Filter Commands Demo
Lines, words and characters in passwd:
1404 1451 65016 passwd

Lines containing the word root:
1:root:x:0:0:root:/root:/bin/bash
10:operator:x:11:0:operator:/root:/sbin/nologin

Count of lines containing root:
2

Lines that do NOT contain root:
bin:x:1:1:bin:/bin:/sbin/nologin
daemon:x:2:2:daemon:/sbin:/sbin/nologin

Replace ':' with '*' in passwd:
root*x*0*0*root*/root*/bin/bash

First column (usernames) of passwd:
root
bin
daemon
```

PROGRAM 6:

Write a shell script to remove the files which has file size as zero bytes.

AIM

To write a shell script that checks whether a given file exists and removes it if its size is zero bytes.

PROCEDURE

STEP 1: Start the process.

STEP 2: Create a file using vi editor.

STEP 3: Get the file name as input from the user.

STEP 4: Use the if..else statement to check whether the file exists using the -e flag.

STEP 5: If the file exists, check its size using the -s flag. If size is zero, remove it using rm.

STEP 6: Display an appropriate message for each case (file removed, file has content, file not found).

STEP 7: Stop the process.

SHELL SCRIPT

```
#!/bin/bash

clear
echo "Enter any file name:"
read filenm          # Read file name from user

if [ -e $filenm ]     # Check if file exists
then
    echo "$filenm file exists"
    if [ -s $filenm ]  # Check if file size > 0
    then
        echo "$filenm file has size > 0"
    else
        rm $filenm     # Remove file if size = 0
        echo "$filenm deleted (size was 0 bytes)"
    fi
else
    echo "$filenm file does not exist"
fi
```

OUTPUT

```
[cs2ax@kgcaslinux ~]$ sh zero_file.sh
```

```
Enter any file name:
```

```
one
```

```
one file exists
```

```
one file has size > 0
```

```
[cs2ax@kgcaslinux ~]$ sh zero_file.sh
```

```
Enter any file name:
```

```
empty
```

```
empty file exists
```

```
empty deleted (size was 0 bytes)
```

```
[cs2ax@kgcaslinux ~]$ sh zero_file.sh
```

```
Enter any file name:
```

```
second
```

```
second file does not exist
```

PROGRAM 7:

Write a shell script to find the sum of the individual digits of a given number.

AIM

To write a shell script that accepts a number from the user and calculates the sum of its individual digits using a while loop.

PROCEDURE

STEP 1: Start the process.

STEP 2: Create a file using vi editor.

STEP 3: Read a number n from the user.

STEP 4: Declare sd=0 to store single digit and sum=0 to accumulate total.

STEP 5: Use a while loop: extract last digit using $n \% 10$, add to sum, then remove last digit using $n / 10$.

STEP 6: Repeat until n becomes 0. Display the sum.

STEP 7: Stop the process.

SHELL SCRIPT

```
#!/bin/bash

echo -n "Enter a number: "
read n

sd=0    # Stores single digit (remainder)
sum=0   # Accumulates sum of digits

while [ $n -gt 0 ]    # Loop until all digits are processed
do
    sd=$(( $n % 10 ))    # Extract last digit
    n=$(( $n / 10 ))     # Remove last digit
    sum=$(( $sum + $sd )) # Add digit to sum
done

echo "Sum of all digits is $sum"
```

OUTPUT

```
[cs2ax@kgcaslinux ~]$ sh digit_sum.sh
Enter a number: 12
Sum of all digits is 3

[cs2ax@kgcaslinux ~]$ sh digit_sum.sh
Enter a number: 4567
Sum of all digits is 22
```

PROGRAM 8:

Write a shell script to find the greatest among the given set of numbers using command line arguments.

AIM

To write a shell script that accepts numbers as command line arguments and finds the greatest and smallest among them.

PROCEDURE

STEP 1: Start the process.

STEP 2: Create a file using vi editor.

STEP 3: Accept count of numbers as first argument (\$1) and read each number into an array using a for loop.

STEP 4: Initialize both 'small' and 'greater' with the first element of the array.

STEP 5: Use a for loop to compare each element; update 'small' if element is smaller, update 'greater' if larger.

STEP 6: Display both the smallest and greatest numbers.

STEP 7: Display the output on screen.

STEP 8: Stop the process.

SHELL SCRIPT

```
#!/bin/bash

clear
for (( i=0; i<$1; i++ ))      # $1 = count of numbers to enter
do
    echo "Enter number $((i+1)):"
    read nos[$i]              # Read each number into array
done

echo "Numbers entered are:"
for (( i=0; i<$1; i++ ))
do
    echo ${nos[$i]}
done

small=${nos[0]}
greater=${nos[0]}

for (( i=0; i<$1; i++ ))      # Compare all numbers
do
    if [ ${nos[$i]} -lt $small ]; then
        small=${nos[$i]}      # Update smallest
    elif [ ${nos[$i]} -gt $greater ]; then
```

```
    greater=${nos[$i]}      # Update greatest
  fi
done

echo "Smallest number in the array: $small"
echo "Greatest number in the array: $greater"
```

OUTPUT

```
[ctb11@Kgcaslinux ~]$ sh greatest.sh 3
Enter number 1:
3
Enter number 2:
98
Enter number 3:
56
Numbers entered are:
3 98 56
Smallest number in the array: 3
Greatest number in the array: 98
```

PROGRAM 9:

Write a shell script for palindrome checking.

AIM

To write a shell script that reads a number from the user and checks whether it is a palindrome by reversing its digits and comparing with the original.

PROCEDURE

STEP 1: Create a file using vi editor.

STEP 2: Read the number n from the user.

STEP 3: Initialize sd=0 (stores remainder) and rev="" (stores reversed number).

STEP 4: Store the original number in a separate variable 'on'.

STEP 5: Use a while loop: extract last digit using $n \% 10$, append to rev, then remove last digit using $n / 10$.

STEP 6: Repeat steps 4-5 until n becomes 0.

STEP 7: Compare the reversed number with the original. If equal, it is a palindrome.

STEP 8: Stop the process.

SHELL SCRIPT

```
#!/bin/bash

clear
echo -n "Enter Number: "
read n                # Read input number

sd=0                  # Stores remainder
rev=""                # Stores reversed number
on=$n                 # Save original number

while [ $n -gt 0 ]
do
    sd=$(( $n % 10 ))    # Get last digit
    n=$(( $n / 10 ))     # Remove last digit
    rev=$(echo ${rev}${sd}) # Append digit to reversed string
done

if [ $on -eq $rev ]    # Compare original with reversed
then
    echo "Number $on is a Palindrome"
else
    echo "Number $on is NOT a Palindrome"
fi
```

OUTPUT

```
[ctb11@Kgcaslinux ~]$ sh palindrome.sh
```

```
Enter Number: 121
```

```
Number 121 is a Palindrome
```

```
[ctb11@Kgcaslinux ~]$ sh palindrome.sh
```

```
Enter Number: 1234
```

```
Number 1234 is NOT a Palindrome
```

PROGRAM 10:

Write a shell script to print the multiplication table of the given argument using for loop.

AIM

To write a shell script that reads a table number and range from the user and prints its multiplication table using a for loop.

PROCEDURE

STEP 1: Create a file using vi editor.

STEP 2: Read the table number n from the user using the read command.

STEP 3: Read the range from the user using the read command.

STEP 4: Use a for loop starting from 1 up to range. Inside the loop calculate the product using expr and display each row.

STEP 5: Stop the process.

SHELL SCRIPT

```
#!/bin/bash

echo "Enter the table number:"
read n

echo "Enter the range:"
read range

echo "Multiplication table for $n up to range $range"

for (( i=1; i<=range; i++ ))
{
    echo " $i X $n = `expr $n \* $i`"  # Loop executes until i <= range
}
```

OUTPUT

```
[csbl@Kgcaslinux ~]$ sh multiplication.sh
Enter the table number:
5
Enter the range:
10
Multiplication table for 5 up to range 10
1 X 5 = 5
2 X 5 = 10
3 X 5 = 15
4 X 5 = 20
5 X 5 = 25
6 X 5 = 30
7 X 5 = 35
8 X 5 = 40
9 X 5 = 45
10 X 5 = 50
```

PROGRAM 11:

Write a shell script to display and analyze group information.

- a. Display all groups available in the system.
- b. Display the group ID of a specified group.
- c. List users belonging to a particular group.
- d. Count the total number of groups available.
- e. Display groups assigned to the current user.

AIM

To write a shell script that displays and analyzes group information: all groups, group ID, group members, total group count, and current user's groups.

PROCEDURE

STEP 1: Create a shell script file using the vi editor.

STEP 2: Use 'cat /etc/group | cut -d: -f1' to display all groups available in the system.

STEP 3: Accept a group name from the user and use 'getent group' to display its group ID.

STEP 4: Accept a group name and use 'getent group' to list all users belonging to that group.

STEP 5: Use 'cat /etc/group | wc -l' to count the total number of groups.

STEP 6: Use the 'groups' command to display all groups assigned to the current user.

STEP 7: Stop the process.

SHELL SCRIPT

```
#!/bin/bash

# a. Display all groups available in the system
echo "--- All Groups in the System ---"
cat /etc/group | cut -d: -f1

# b. Display the group ID of a specified group
echo ""
echo "Enter group name to find its ID:"
read grpname
grp_id=$(getent group $grpname | cut -d: -f3)
echo "Group ID of '$grpname' is: $grp_id"

# c. List users belonging to a particular group
echo ""
echo "Enter group name to list its members:"
read grpname2
echo "Users in group '$grpname2':"
getent group $grpname2 | cut -d: -f4 | tr ',' '\n'

# d. Count total number of groups
echo ""
```

```
totalgroups=$(cat /etc/group | wc -l)
echo "Total number of groups: $totalgroups"

# e. Display groups assigned to current user
echo ""
echo "Groups assigned to current user ($USER):"
groups $USER
```

OUTPUT

```
[csb1@Kgcaslinux ~]$ sh group_info.sh
--- All Groups in the System ---
root
bin
daemon
users ...

Enter group name to find its ID:
users
Group ID of 'users' is: 100

Enter group name to list its members:
users
Users in group 'users':
csb1
csb2

Total number of groups: 72

Groups assigned to current user (csb1):
csb1 users wheel
```

PROGRAM 12:

Write a shell script to monitor process activities.

- a. Display all running processes.
- b. Display processes belonging to the current user.
- c. Show top five CPU consuming processes.
- d. Display process ID and parent process ID.
- e. Display the total number of running processes.

AIM

To write a shell script that monitors process activities: all processes, user processes, top CPU consumers, PID/PPID details, and total process count.

PROCEDURE

STEP 1: Create a shell script file using the vi editor.

STEP 2: Use 'ps -ef' to display all running processes in the system.

STEP 3: Use 'ps -u \$USER' to display processes belonging to the current user only.

STEP 4: Use 'ps -eo pid,comm,%cpu --sort=-%cpu | head -6' to show top 5 CPU consuming processes.

STEP 5: Use 'ps -eo pid,ppid,comm' to display process ID and parent process ID.

STEP 6: Use 'ps -ef | wc -l' to count total number of running processes.

STEP 7: Stop the process.

SHELL SCRIPT

```
#!/bin/bash

# a. Display all running processes
echo "--- All Running Processes ---"
ps -ef

# b. Processes belonging to the current user
echo ""
echo "--- Processes of Current User ($USER) ---"
ps -u $USER

# c. Top 5 CPU consuming processes
echo ""
echo "--- Top 5 CPU Consuming Processes ---"
echo "PID  COMMAND      %CPU"
ps -eo pid,comm,%cpu --sort=-%cpu | head -6

# d. Process ID and Parent Process ID
echo ""
echo "--- PID and PPID ---"
echo "PID  PPID  COMMAND"
```

```
ps -eo pid,ppid,comm | head -15
```

```
# e. Total number of running processes
echo ""
total=$(ps -ef | wc -l)
echo "Total number of running processes: $total"
```

OUTPUT

```
[csbl@Kgcaslinux ~]$ sh process_monitor.sh
--- All Running Processes ---
UID  PID  PPID  C  STIME TTY   TIME   CMD
root   1    0  0  09:00 ?    00:00:02 /sbin/init
root   2    0  0  09:00 ?    00:00:00 [kthreadd] ...

--- Processes of Current User (csbl) ---
PID  TTY   TIME   CMD
1234 pts/0  00:00:00 bash

--- Top 5 CPU Consuming Processes ---
PID  COMMAND      %CPU
1024  firefox      12.5
1100  Xorg          5.2
1200  python3       3.1
1300  bash          0.5
1400  ps            0.2

--- PID and PPID ---
PID  PPID  COMMAND
1    0     systemd
2    0     kthreadd

Total number of running processes: 185
```

PROGRAM 13:

Write a shell script to display network configuration information.

- a. Display system hostname.
- b. Display IP address of all network interfaces.
- c. Display routing table information.
- d. Display DNS server configuration.
- e. Test network connectivity with a remote host.

AIM

To write a shell script that displays network configuration details: hostname, IP addresses, routing table, DNS servers, and tests network connectivity.

PROCEDURE

STEP 1: Create a shell script file using the vi editor.

STEP 2: Use 'hostname' command to display the system hostname.

STEP 3: Use 'ip addr show' to display IP addresses of all network interfaces.

STEP 4: Use 'ip route' to display the routing table information.

STEP 5: Use 'cat /etc/resolv.conf' to display DNS server configuration.

STEP 6: Accept a remote host from the user and use 'ping -c 4' to test connectivity.

STEP 7: Stop the process.

SHELL SCRIPT

```
#!/bin/bash

# a. Display system hostname
echo "--- System Hostname ---"
hostname

# b. IP address of all network interfaces
echo ""
echo "--- IP Addresses of Network Interfaces ---"
ip addr show | grep -E 'inet |^[0-9]+:'

# c. Routing table information
echo ""
echo "--- Routing Table ---"
ip route

# d. DNS server configuration
echo ""
echo "--- DNS Server Configuration ---"
cat /etc/resolv.conf

# e. Test network connectivity
```

```
echo ""  
echo "Enter remote host to test (e.g. 8.8.8.8):"  
read remotehost  
echo "Testing connectivity to $remotehost ..."  
ping -c 4 $remotehost
```

OUTPUT

```
[csb1@Kgcaslinux ~]$ sh network_info.sh  
--- System Hostname ---  
Kgcaslinux  
  
--- IP Addresses of Network Interfaces ---  
1: lo:  inet 127.0.0.1/8 scope host lo  
2: eth0: inet 192.168.1.100/24 brd 192.168.1.255  
  
--- Routing Table ---  
default via 192.168.1.1 dev eth0  
192.168.1.0/24 dev eth0 proto kernel  
  
--- DNS Server Configuration ---  
nameserver 8.8.8.8  
nameserver 8.8.4.4  
  
Enter remote host to test (e.g. 8.8.8.8):  
8.8.8.8  
Testing connectivity to 8.8.8.8 ...  
64 bytes from 8.8.8.8: icmp_seq=1 ttl=118 time=12.3 ms  
4 packets transmitted, 4 received, 0% packet loss
```

PROGRAM 14:

Write a shell script to monitor system logs and detect suspicious activity.

- a. Display recent system log entries.
- b. Display login history of users.
- c. Display failed login attempts.
- d. Search for specific keywords in log files.
- e. Display the last 10 security related log messages.

AIM

To write a shell script that monitors system logs to detect suspicious activity: recent logs, login history, failed logins, keyword search, and security messages.

PROCEDURE

STEP 1: Create a shell script file using the vi editor.

STEP 2: Use 'tail -20 /var/log/messages' or 'journalctl -n 20' to display recent log entries.

STEP 3: Use 'last | head -15' to display login history of users.

STEP 4: Use grep to search for 'Failed password' in /var/log/secure to find failed login attempts.

STEP 5: Accept a keyword from the user and search for it in log files using grep.

STEP 6: Use 'tail -10 /var/log/secure' to display the last 10 security-related log messages.

STEP 7: Stop the process.

SHELL SCRIPT

```
#!/bin/bash

# a. Recent system log entries
echo "--- Recent System Log Entries ---"
tail -20 /var/log/messages 2>/dev/null || journalctl -n 20

# b. Login history of users
echo ""
echo "--- Login History of Users ---"
last | head -15

# c. Failed login attempts
echo ""
echo "--- Failed Login Attempts ---"
grep "Failed password" /var/log/secure 2>/dev/null | tail -10

# d. Search for specific keyword in log files
echo ""
echo "Enter keyword to search in /var/log/messages:"
read keyword
echo "Results for '$keyword':"
grep -i "$keyword" /var/log/messages 2>/dev/null | tail -10
```

```
# e. Last 10 security related log messages
echo ""
echo "--- Last 10 Security Log Messages ---"
tail -10 /var/log/secure 2>/dev/null || journalctl -u sshd -n 10
```

OUTPUT

```
[csb1@Kgcaslinux ~]$ sh log_monitor.sh
--- Recent System Log Entries ---
Jun  3 09:00:01 Kgcaslinux systemd: Started Session 5 of user csb1.
Jun  3 09:01:15 Kgcaslinux kernel: eth0: link is up ...

--- Login History of Users ---
csb1 pts/0 192.168.1.10 Mon Jun 3 09:00 still logged in
root pts/1 192.168.1.5  Mon Jun 3 08:45 - 08:55 (00:10)

--- Failed Login Attempts ---
Jun 3 08:30 sshd: Failed password for invalid user admin from 10.0.0.1

Enter keyword to search in /var/log/messages:
error
Results for 'error':
Jun 3 09:02 kernel: ACPI Error: Method parse/execution failed

--- Last 10 Security Log Messages ---
Jun 3 09:00 sshd: Accepted password for csb1 from 192.168.1.10
```

PROGRAM 15:

Write a shell script to analyze disk usage.

- a. Display disk usage of all directories in the home folder.
- b. Display the top 5 largest directories in the system.
- c. Display the number of files in each directory.
- d. Display the file system type of each partition.
- e. Display free disk space available.

AIM

To write a shell script that analyzes disk usage: home directory sizes, top 5 largest directories, file counts, filesystem types, and free disk space.

PROCEDURE

STEP 1: Create a shell script file using the vi editor.

STEP 2: Use 'du -sh ~/*' to display disk usage of all directories inside the home folder.

STEP 3: Use 'du -sh /* | sort -rh | head -5' to find the top 5 largest directories in the system.

STEP 4: Use a for loop with 'ls | wc -l' inside each subdirectory to count the number of files.

STEP 5: Use 'df -T' to display the filesystem type of each mounted partition.

STEP 6: Use 'df -h' to display free disk space available on all partitions.

STEP 7: Stop the process.

SHELL SCRIPT

```
#!/bin/bash

# a. Disk usage of all directories in home folder
echo "--- Disk Usage of Home Directory ---"
du -sh ~/* 2>/dev/null

# b. Top 5 largest directories in the system
echo ""
echo "--- Top 5 Largest Directories ---"
du -sh /* 2>/dev/null | sort -rh | head -5

# c. Number of files in each home subdirectory
echo ""
echo "--- File Count in Each Home Subdirectory ---"
for dir in ~/*/
do
    count=$(ls "$dir" 2>/dev/null | wc -l)
    echo "$dir : $count files"
done

# d. Filesystem type of each partition
echo ""
```

```
echo "--- Filesystem Type of Each Partition ---"
df -T | awk '{print $1, $2, $NF}'

# e. Free disk space available
echo ""
echo "--- Free Disk Space Available ---"
df -h
```

OUTPUT

```
[csb1@Kgcaslinux ~]$ sh disk_usage.sh
--- Disk Usage of Home Directory ---
4.0K  /home/csb1/Desktop
8.5M  /home/csb1/Documents
120M  /home/csb1/Downloads

--- Top 5 Largest Directories ---
12G   /usr
5.2G  /var
3.8G  /home
1.5G  /opt
980M  /boot

--- File Count in Each Home Subdirectory ---
/home/csb1/Desktop/ : 3 files
/home/csb1/Documents/ : 12 files
/home/csb1/Downloads/ : 45 files

--- Filesystem Type of Each Partition ---
Filesystem  Type  Mounted on
/dev/sda1   ext4  /
tmpfs       tmpfs /tmp

--- Free Disk Space Available ---
Filesystem  Size  Used Avail Use% Mounted on
/dev/sda1   50G   22G   26G   46% /
tmpfs       3.9G   0   3.9G   0% /tmp
```